Chapter 3: Search in Artificial Intelligence

Prof. Ahmed Guessoum
Course: Introduction to AI

---

1. Problem-Solving Agents

A **problem-solving agent** is an entity that acts based on a well-defined goal. It follows a structured approach:

1. **Goal Formulation**: Defining what the agent wants to achieve. The agent determines its desired outcome based on the environment and its performance measure.
2. **Problem Formulation**: Deciding which actions and states to consider. This involves defining the initial state, available actions, transition model, goal test, and path cost function.
3. **Search**: Finding a sequence of actions that lead to a solution. The agent evaluates possible sequences of actions that transition it from the initial state to the goal state.
4. **Execution**: Carrying out the action sequence to reach the goal state. The agent implements the identified solution and monitors the outcome.

A well-defined **problem** consists of:

- **The Initial State**: Where the agent starts.
- **A Set of Possible Actions**: What actions the agent can take. Each action moves the agent from one state to another.
- **A Transition Model**: How each action affects the state. This determines the resulting state after an action is applied.
- **The Goal Test**: Determines if a state is a goal state. This can be a simple condition (e.g., reaching a destination) or complex (e.g., solving a puzzle).
- **A Path Cost Function**: Assigns a numeric cost to each possible path. It helps evaluate the quality of different solutions by comparing their total costs.

The objective is to find an **optimal solution**, meaning the action sequence with the lowest path cost among all solutions.

1.1 Example Problem: Traveling in Romania

- **Initial state**: The agent is in Arad.
- **Actions**: Move to a neighboring city (e.g., Go(Sibiu), Go(Timișoara), Go(Zerind)).
- **Transition Model**: Defines the result of an action (e.g., RESULT(In(Arad), Go(Zerind)) = In(Zerind)).

- **Goal Test**: Check if the agent is in Bucharest.
- **Path Cost**: Defined as the total distance traveled.
- **Optimal Solution**: The shortest path from Arad to Bucharest with minimal travel cost.

A problem can be **abstracted** by removing unnecessary details (e.g., road conditions, weather) while keeping the essential components. This ensures computational efficiency and allows focusing on critical decision-making aspects.

---

2. Well-Defined Problems & Solutions

2.1 Formal Definition of a Problem

A problem is defined by:

- **Initial State**: The starting state from which the agent begins its journey.
- **Actions**: The set of operations the agent can perform to transition between states.
- **Transition Model**: The function describing the outcome of performing an action in a given state.
- **State Space**: The collection of all possible states reachable from the initial state through valid actions.
- **Goal Test**: The condition that determines whether the current state meets the desired objective.
- **Path Cost Function**: A numerical measure evaluating the cost associated with a given path from the initial state to the goal state. Common cost metrics include distance, time, or resource consumption.

2.2 Solution to a Problem

- A **solution** is an action sequence that leads from the initial state to a goal state.
- The **quality** of a solution is measured by the path cost function, which provides an objective way to compare different paths.
- An **optimal solution** has the lowest path cost among all possible solutions, ensuring minimal resource utilization.

---

3. Example Problems

3.1 Vacuum World

- **States**: Agent's location and the cleanliness of different locations.
- **Actions**: Move Left, Move Right, or Suck dirt.
- **Goal Test**: All squares are clean.
- **Path Cost**: Each step costs 1, encouraging efficient movement.

3.2 8-Puzzle Problem

- **States**: The arrangement of eight numbered tiles and a blank tile in a 3x3 grid.
- **Actions**: Move the blank tile Left, Right, Up, or Down.
- **Goal Test**: The tiles must match the correct configuration.
- **Path Cost**: Each move incurs a cost of 1.

3.3 8-Queens Problem

- **States**: The positions of 0 to 8 queens on a chessboard.
- **Actions**: Add a queen to a valid position.
- **Goal Test**: No two queens should attack each other.
- **Better Formulation**: Only place queens in non-attacking positions to reduce the search space.

---

4. Searching for Solutions

4.1 Search Tree

- Represents possible action sequences starting at the initial state.
- **Nodes**: Represent states.
- **Edges**: Represent actions.
- **Frontier**: The set of unexpanded leaf nodes.

4.2 Search Algorithms

- Vary based on how they choose which state to expand next.
- **Loopy Paths**: Paths that contain repeated states can be avoided.
- **Tree-Search vs. Graph-Search**: Graph-Search keeps track of explored states to avoid redundant paths.

---

5. Uninformed Search Strategies

5.1 Breadth-First Search (BFS)

- Expands the shallowest nodes first.
- **Completeness**: Yes.
- **Optimality**: Yes, if all step costs are equal.
- **Time Complexity**: O(b^d).
- **Space Complexity**: O(b^d).

5.2 Uniform-Cost Search (UCS)

- Expands the node with the lowest path cost.
- **Completeness**: Yes.
- **Optimality**: Yes.
- **Time Complexity**: O(b^(1+C∗/ϵC^*/\epsilon)).

5.3 Depth-First Search (DFS)

- Expands the deepest node first.
- **Completeness**: No (unless graph-search).
- **Optimality**: No.
- **Time Complexity**: O(b^m).
- **Space Complexity**: O(bm).

5.4 Depth-Limited Search (DLS)

- DFS with a pre-set depth limit **l**.
- **Completeness**: Yes, if the solution is within the depth limit.
- **Optimality**: No, may miss solutions beyond the limit.
- **Time Complexity**: O(b^l).
- **Space Complexity**: O(bl).

5.5 Iterative Deepening Search (IDS)

- Repeatedly applies DLS, increasing the depth limit at each step.
- **Completeness**: Yes.
- **Optimality**: Yes, if step costs are equal.
- **Time Complexity**: O(b^d) (similar to BFS).
- **Space Complexity**: O(bd) (like DFS but with better memory efficiency).
- **Preferred** for large search spaces when the solution depth is unknown.

5.6 Bidirectional Search

- Runs two simultaneous searches:
    - One from the initial state forward.
    - One from the goal state backward.
- **Completeness**: Yes.
- **Optimality**: Yes.
- **Time Complexity**: O(b^(d/2)) + O(b^(d/2)), much better than BFS O(b^d).
- **Requires** finding predecessor states, which may be complex.

Comparison of Uninformed Search Strategies

Search Strategy	Complete?	Optimal?	Time Complexity	Space Complexity
BFS	Yes	Yes (if uniform cost)	$O(b^d)$	$O(b^d)$
UCS	Yes	Yes	$O(b^{1+\frac{C^*}{\epsilon}})$	$O(b^{1+\frac{C^*}{\epsilon}})$
DFS	No	No	$O(b^m)$	$O(bm)$
DLS	No	No	$O(b^l)$	$O(bl)$
IDS	Yes	Yes	$O(b^d)$	$O(bd)$
Bidirectional	Yes	Yes	$O(b^{d/2})$	$O(b^{d/2})$

6. Informed (Heuristic) Search Strategies

6.1 Greedy Best-First Search

- Expands the node that is closest to the goal, as estimated by a heuristic function h(n).
- **Heuristic Function**: h(n) estimates the cost from n to the goal.
- **Completeness**: No (can get stuck in loops).
- **Optimality**: No.
- **Time Complexity**: O(b^m), but can be much better with a good heuristic.
- **Space Complexity**: O(b^m).

6.2 A* Search

A* Search combines two key components to evaluate nodes:

- **g(n)**: The actual cost to reach the node from the start state.
- **h(n)**: The estimated cost to reach the goal from the node (heuristic).
- The evaluation function is:
  $f(n)=g(n)+h(n)$ $f(n) = g(n) + h(n)$
  where $f(n)$ $f(n)$ is the estimated cost of the cheapest solution through node nn.

A* is similar to **Uniform-Cost Search (UCS)**, but instead of using only $g(n)$ $g(n)$, it uses $f(n)=g(n)+h(n)$ $f(n) = g(n) + h(n)$ to prioritize nodes.

Conditions for Optimality

- **Admissibility**: A heuristic $h(n)$ $h(n)$ is **admissible** if it never overestimates the true cost to reach the goal. This ensures that $f(n)$ $f(n)$ never overestimates the true cost of a solution through nn.

    - Example: **Straight-Line Distance (SLD)** is admissible because it never overestimates the actual distance.
- **Consistency (Monotonicity)**: A heuristic h(n)h(n) is **consistent** if for every node nn and its successor n′n', the following holds:
  h(n)≤c(n,a,n′)+h(n′)h(n) \leq c(n, a, n') + h(n')
  where c(n,a,n′)c(n, a, n') is the cost of taking action aa from nn to n′n'.

    - Consistency is required for **graph search** to ensure optimality.
    - Every consistent heuristic is also admissible, but not all admissible heuristics are consistent.

Optimality of A* Search

- **Tree-Search Version**: A* is optimal if h(n)h(n) is **admissible**.
- **Graph-Search Version**: A* is optimal if h(n)h(n) is **consistent**.
- **Proof**: A* expands all nodes with f(n)<C∗f(n) < C^* (where C∗C^* is the cost of the optimal solution) and may expand some nodes with f(n)=C∗f(n) = C^* before selecting the goal node.
- **Pruning**: A* can safely ignore nodes with f(n)>C∗f(n) > C^*, which helps in reducing the search space.

Properties of A* Search

- **Completeness**: A* is complete if all step costs exceed some finite ϵ\epsilon and the branching factor bb is finite.
- **Optimal Efficiency**: A* is optimally efficient for any given consistent heuristic, meaning no other optimal algorithm is guaranteed to expand fewer nodes than A*.
- **Time Complexity**: For problems like the **8-puzzle**, the time complexity is O(bd)O(b^d), where bb is the branching factor and dd is the solution depth.
- **Space Complexity**: A* keeps all generated nodes in memory, which can lead to high memory usage, often causing it to run out of space before time.

Memory-Bounded Heuristic Search

- *Iterative-Deepening A (IDA)**: A memory-efficient variant of A* that uses iterative deepening with a cutoff based on the ff-cost (i.e., f(n)=g(n)+h(n)f(n) = g(n) + h(n)) rather than depth.
    - IDA* is practical for problems with **unit step costs**.
- **Recursive Best-First Search (RBFS)**: Mimics standard best-first search but uses only linear space. It uses an ff-limit variable to track the best alternative path and unwinds recursion if the current path exceeds this limit.
    - RBFS is more efficient than IDA* but suffers from excessive node regeneration due to "mind changes."
    - **Optimality**: RBFS is optimal if the heuristic h(n)h(n) is admissible.
    - **Space Complexity**: Linear in the depth of the deepest optimal solution.
    - **Time Complexity**: Potentially exponential, depending on the heuristic's accuracy and how often the best path changes.

Learning to Search Better

- **Meta-Level State Space**: A concept where each state captures the internal computational state of a program searching in an object-level state space (e.g., searching for a route in Romania).
    - The internal state of A* consists of the current search tree.
    - Each action in the meta-level state space is a computation step that alters the internal state (e.g., expanding a leaf node in A*).
- **Learning**: A meta-level learning algorithm can learn from past search experiences to avoid exploring unpromising subtrees, thereby minimizing the total cost of problem-solving by trading off computational expense and path cost.

Heuristic Functions

- **8-Puzzle Example**: The 8-puzzle is a classic heuristic search problem with a branching factor of ~3 and an average solution cost of 22 steps.
    - An exhaustive tree search to depth 22 would explore ~3.1 x 10^10 states.
    - With **graph search**, the number of states is reduced to 9!/2 = 181,440 distinct states.
- **15-Puzzle**: The number of distinct states is much larger, making heuristic search essential.

Heuristics for 15-Puzzle

- **Admissible Heuristics**:
    - **h_1**: The number of misplaced tiles.
    - **h_2**: The sum of the distances of the tiles from their goal positions (Manhattan distance).
    - Both h1h_1 and h2h_2 are admissible because they never overestimate the true cost.

Heuristic Accuracy & Performance

- **Effective Branching Factor (b∗b^*)**: A measure of heuristic quality. If A* generates NN nodes to find a solution at depth dd, then:
  N+1=1+b∗+(b∗)2+⋯+(b∗)dN + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d
    - For a well-designed heuristic, b∗≈1b^* \approx 1, allowing large problems to be solved efficiently.
- **Comparison of h1h_1 and h2h_2**:

    - h2h_2 (Manhattan distance) is better than h1h_1 (misplaced tiles) and far better than **Iterative Deepening Search (IDS)**.